

Dancing Pipes

Programmierprojekt
Sommersemester 2024

Atila Özdemir

Kirill Dedov

Matsvei Tsiareshka

Toni Kostov

July 24, 2024

1 So macht man eine Section

Hier kann man text schreiben.

So macht man eine neue Zeile. So kann man Code im Text schreiben: zum Beispiel `int a = 5;`.

1.1 So macht man eine Subsection

Wenn man mehrere Zeilen Code schreiben will nutzt man das:

```
public static void changeChord() {  
    sequencer.stop();  
    sequencer.setSequence(sequence);  
    sequencer.start();  
}
```

So macht man eine Subsection ohne abzählen

Tabellen kann man auch machen:

Spalte1	Spalte 2
hier kann man text schreiben	und hier auch
noch Text schreiben	und hier auch mehr text
Spalten kann man auch so mergen	
und hier	weiter machen
Zeilen kann man auch mergen	das geht so und dann so
danach	einfach weiter machen

Wenn man eine Liste machen will, macht man das so:

- Element 1
- Element 2
- Element 3
- Element 4

Viel Spaß :)

2 Introduction

This document outlines a project that explores the intersection of human movement and musical expression. The primary objective is to develop an interactive system that translates a user's physical movements into musical notes played on a digital pipe organ. This project can be broadly divided into two key stages:

- Capturing and Interpreting of Movements
- Transmission of Motion Data to Digital Pipe Organ as MIDI Signal

3 Elaboration on Stages

3.1 Model Selection for Capturing and Interpreting of Movements

3.1.1 Kinect

The project initially explored the use of the Microsoft Kinect device for capturing and interpreting user movements. The Kinect offered one main advantage:

- 3D Data Capture: The Kinect's dual camera system and various sensors allowed for capturing 3D positional data of the user's body, providing rich information for movement analysis.

However, significant challenges arose during development:

- Limited Compatibility: The Kinect device's reliance on Windows operating systems posed an obstacle for team members using different platforms.
- Accessibility Issues: The discontinuation of Kinect production made acquiring the device difficult. The team faced limitations in providing device support for concurrent testing across multiple team members. This significantly hampered development speed.

Due to the accessibility limitations for both the development team and potential users, the team decided to investigate more modern, dynamic, and user-friendly solutions for capturing and interpreting movements.

3.1.2 Webcam-Based Pose Estimation with MoveNet Using Machine Learning

Following the exploration of the Kinect device, the project pivoted towards investigating the capabilities of artificial intelligence-supported models for capturing and interpreting movements. This shift offered several advantages:

- Accessibility: Webcams are readily available on most personal computers, eliminating the need for additional sensors or specialized equipment. This significantly reduces barriers to both development and user adoption.
- Cross-platform Compatibility: Webcam functionality is inherent to most operating systems, ensuring broader compatibility compared to the Windows-specific Kinect.
- Development Efficiency: The widespread availability of webcams allows each team member to conduct independent testing and development cycles, accelerating the iterative process. Given these advantages, the team decided to explore MoveNet's Lightning model as the first approach. This selection leveraged the TensorFlow and OpenPose libraries, offering a well-established framework for pose estimation tasks.

While the initial tests with MoveNet demonstrated promising results in detecting the movements of a seated user, the model exhibited limitations:

- Limited Posture Recognition: MoveNet struggled to accurately perceive the movements of a standing person with the same efficacy as those in a seated position. Extensive testing across various environments (lighting conditions, background variations) and user demographics (gender, height, skin tone) failed to yield significant improvements in this area.
- Performance Issues with Speed: The model's processing speed proved to be a bottleneck, particularly when dealing with fast movements or movements performed at low angles. These situations resulted in stuttering, slowdowns, and instances of missed detections.

Due to these limitations, the project team decided to pursue alternative models. The primary focus shifted towards finding a model that offered:

- **Lightweight Architecture:** A smaller, more efficient model would improve processing speed and address the performance issues encountered with MoveNet.
- **Posture Agnostic Detection:** The chosen model should be capable of accurately detecting and interpreting human movements regardless of the user's posture (standing or sitting).

By addressing these shortcomings, the team aimed to establish a more robust and versatile solution for capturing and interpreting user movements.

3.1.3 Webcam-Based Pose Estimation with MediaPipe Using Machine Learning

Following the limitations encountered with MoveNet, the project explored MediaPipe as a potential alternative for capturing and interpreting user movements. MediaPipe shares some similarities with MoveNet in its model architecture, but offers distinct advantages:

- **Simplified Installation:** MediaPipe boasts a streamlined installation process compared to MoveNet, reducing development setup time and streamlining project workflow.
- **Enhanced Performance:** Our evaluation revealed that MediaPipe outperformed MoveNet in terms of processing speed, frame capture rate, and overall detection accuracy. This translates to smoother user experience with minimal latency and improved responsiveness to user movements.

However, during the evaluation process, one potential drawback of MediaPipe was identified:

- **Limited Customization:** While MediaPipe offers pre-built solutions for various tasks, its customizability for highly specialized applications might be less extensive compared to frameworks like TensorFlow.

Considering the project's time constraints and the positive results from the performance evaluation, MediaPipe was chosen as the preferred model for further development. However, the project acknowledges the need for potential future adjustments to address any limitations in customization or potential biases within the model.

3.2 Transmission of Motion Data to Digital Pipe Organ as MIDI Signal

The identified movements will be mapped to corresponding MIDI messages. This mapping assigns specific movements to control various musical parameters like pitch, volume, and timbre (organ stop selection). A software application will be developed to translate the interpreted movements into MIDI signals in real-time. This software will act as an intermediary, receiving movement data from the sensors and transmitting the corresponding MIDI messages to the digital pipe organ.